

# 一、Conda 配置

## 1. \*\*下载安装程序\*\*

优先推荐访问[清华大学开源软件镜像站](<https://mirrors.tuna.tsinghua.edu.cn/anaconda/miniconda/>), 以获得更快的下载速度。根据你的系统选择最新的`.exe`版本下载。

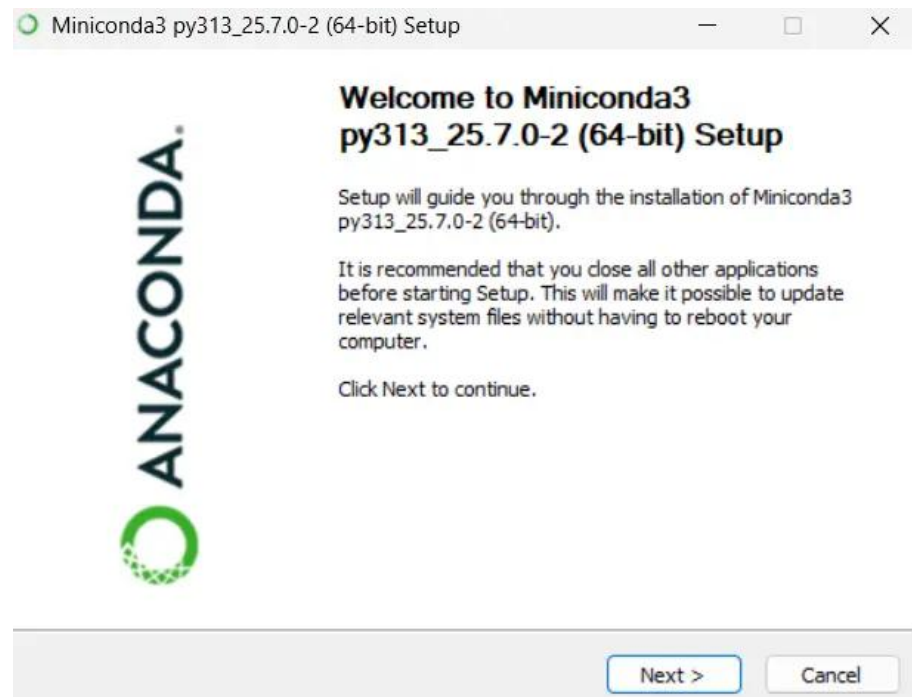
![选择 Miniconda 版本](images/1\_2\_15.webp)

你也可以从 [Miniconda 官方网站](<https://docs.conda.io/en/latest/miniconda.html>)下载。

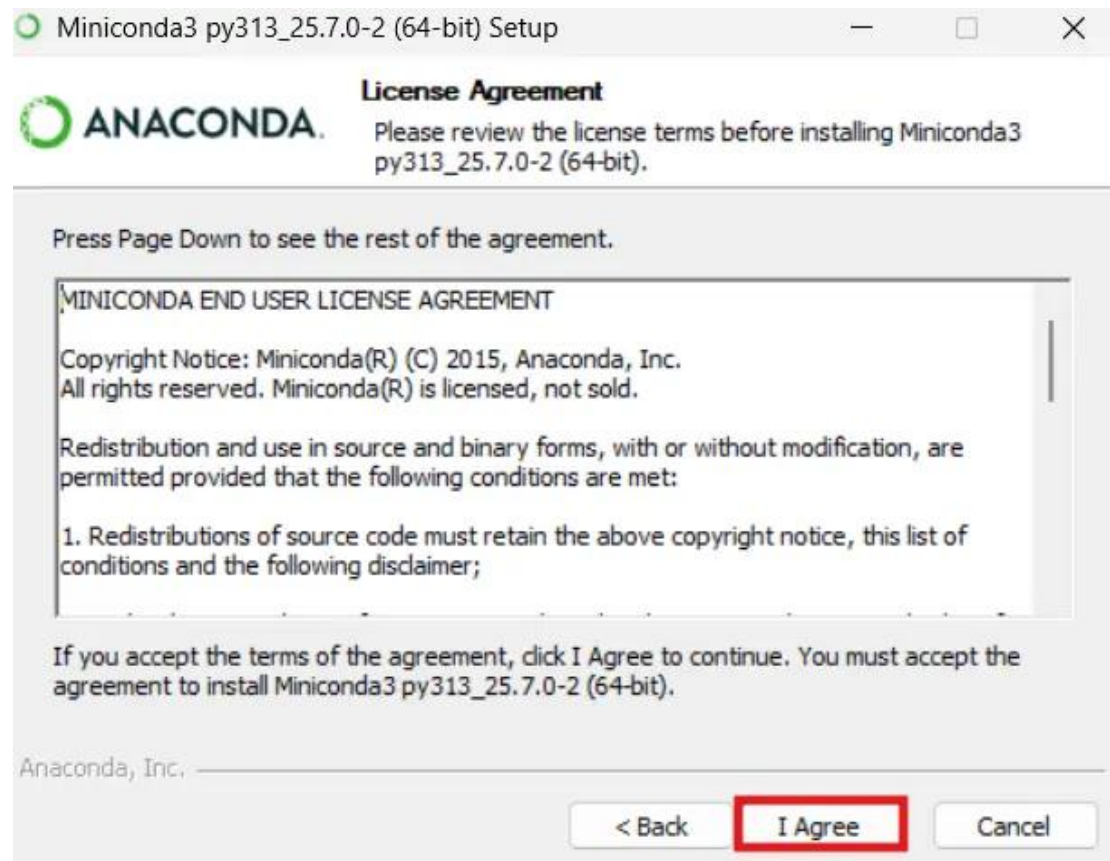
## 2. \*\*运行安装向导\*\*

下载完成后, 双击`.exe`文件启动安装。按照向导提示操作:

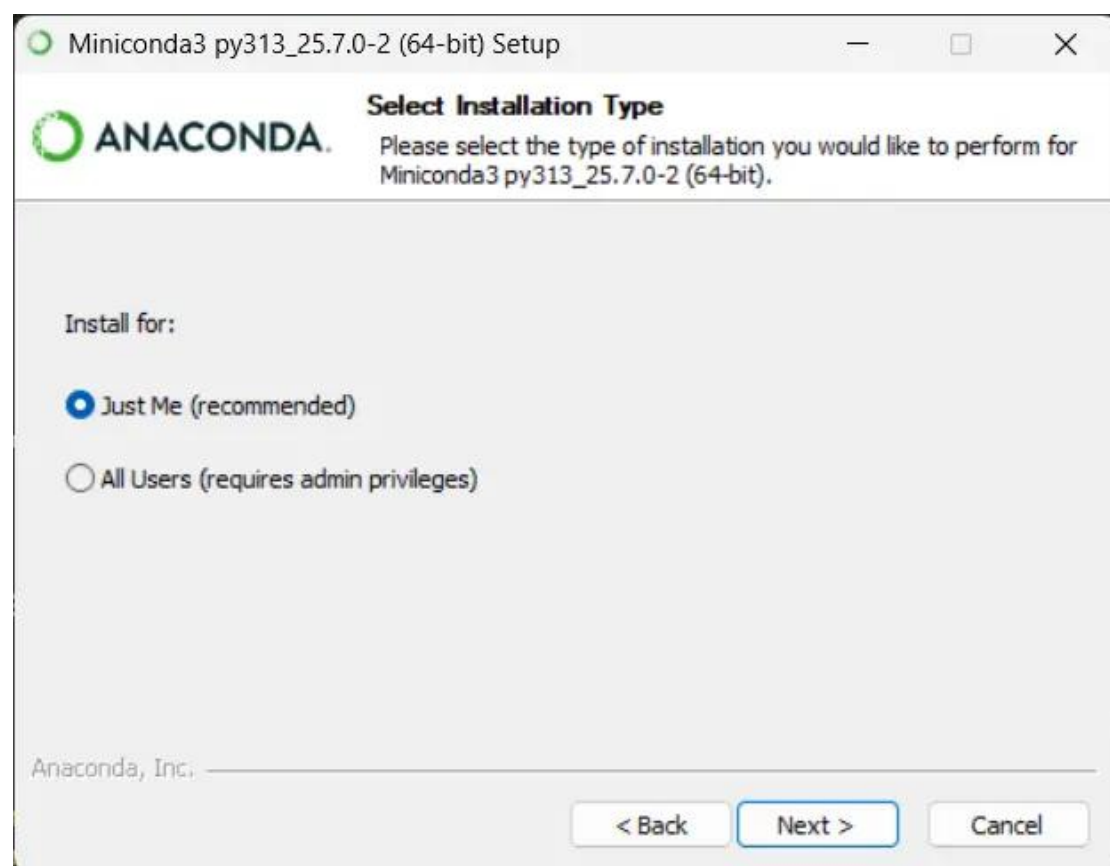
\* \*\*Welcome\*\*: 点击`Next`。



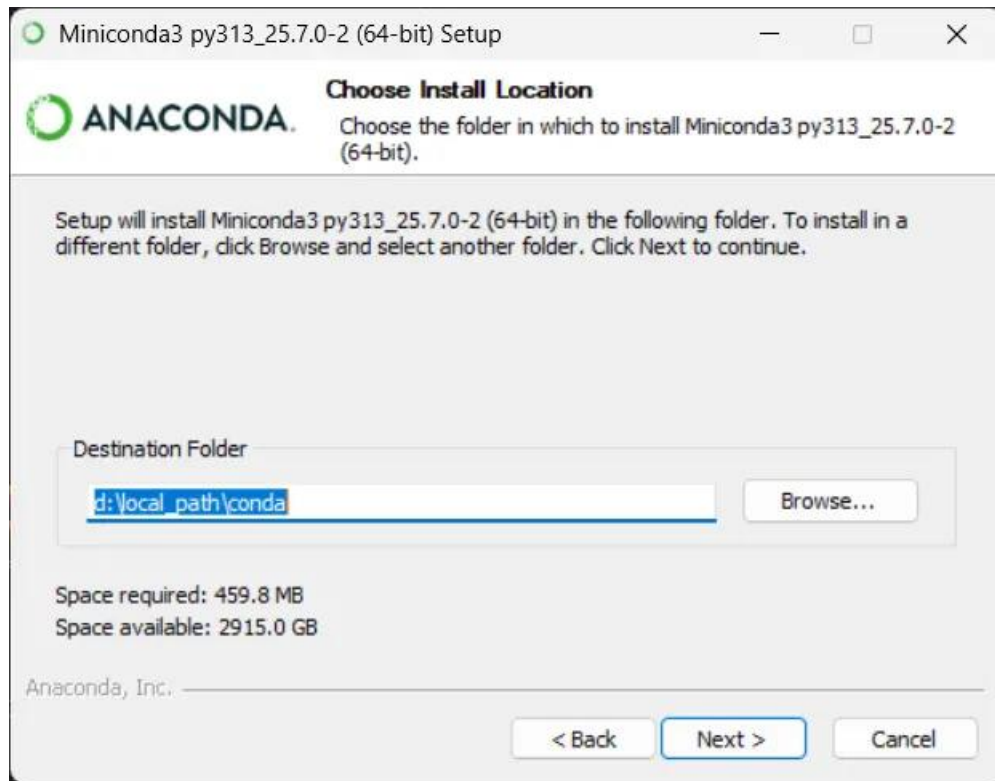
\* \*\*License Agreement\*\*: 点击`I Agree`。



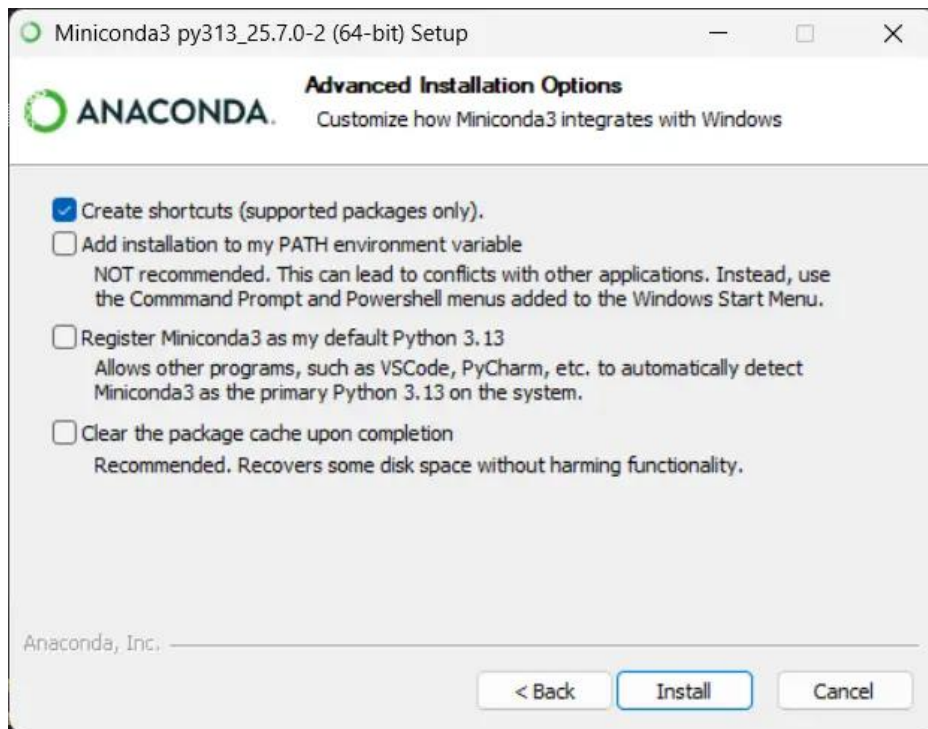
\* \*\*Installation Type\*\*: 选择 `Just Me`, 点击 `Next`。



\* **\*\*Choose Install Location\*\***: 建议保持默认路径, 或选择一个不含中文和空格的路径。点击 `Next`。



\* **\*\*Advanced Installation Options\*\***: **\*\*请不要勾选\*\*** “Add Miniconda3 to my PATH environment variable” 。我们将稍后手动配置环境变量。点击 `Install`。

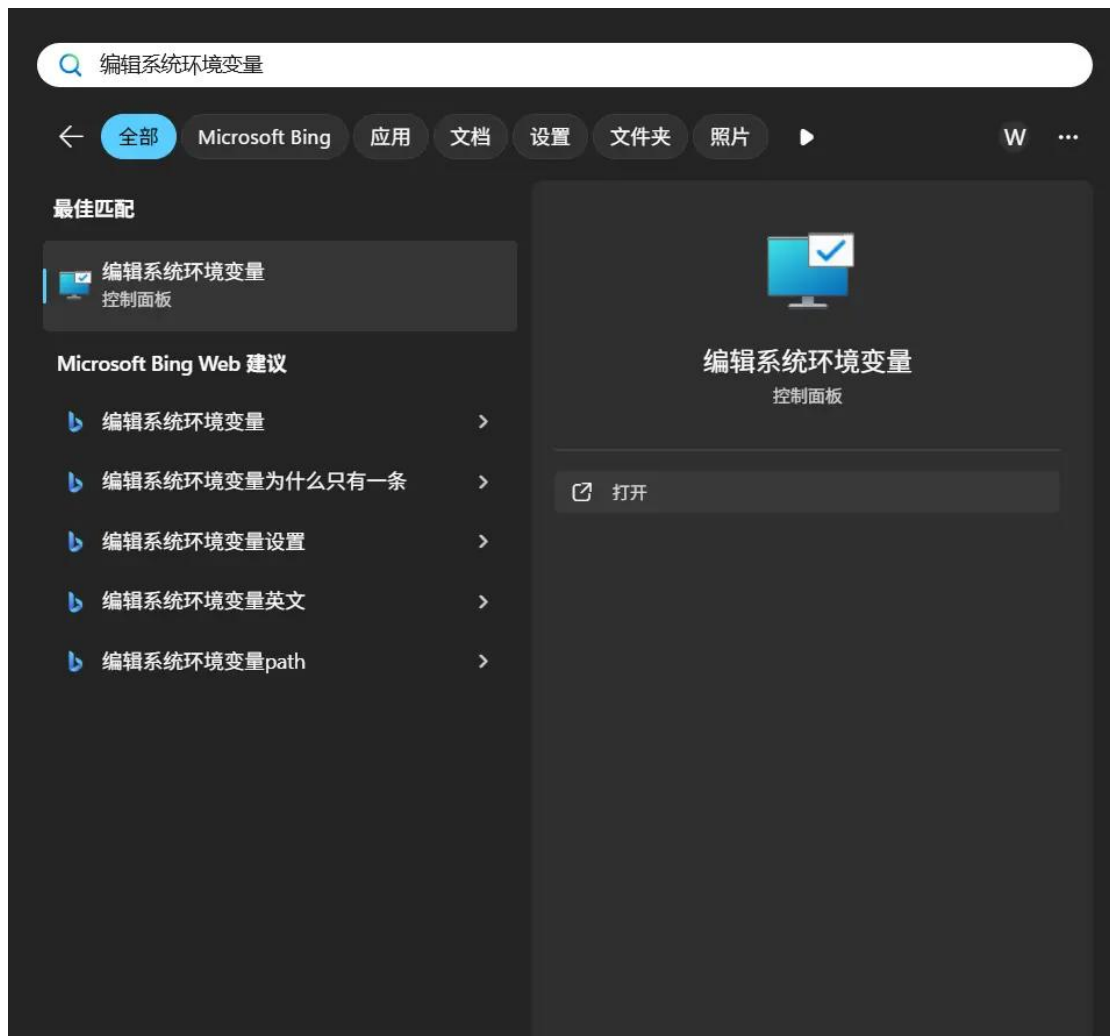


\* **\*\*Installation Complete\*\***: 安装完成后，点击 `Next`，然后取消勾选 “Learn more” 并点击 `Finish` 完成安装。

### 3. **\*\*手动配置环境变量\*\***

为了能在任意终端窗口使用 `conda` 命令，需要手动配置环境变量。

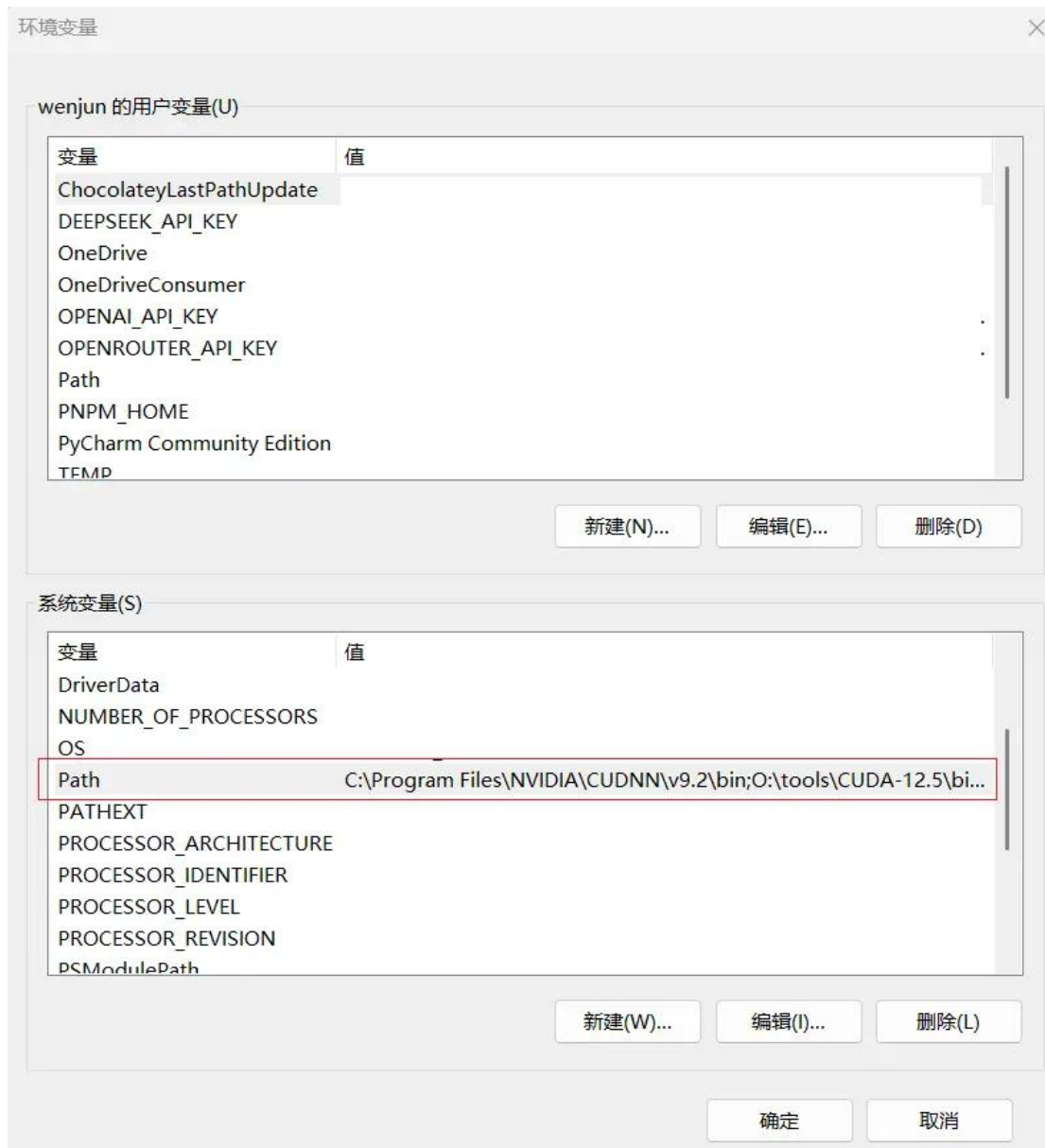
\* 在 Windows 搜索栏中搜索 “编辑系统环境变量” 并打开。



\* 在“系统属性”窗口中，点击“环境变量”。



\* 在“环境变量”窗口中,找到“系统变量”下的`Path`变量,选中并点击“编辑”。

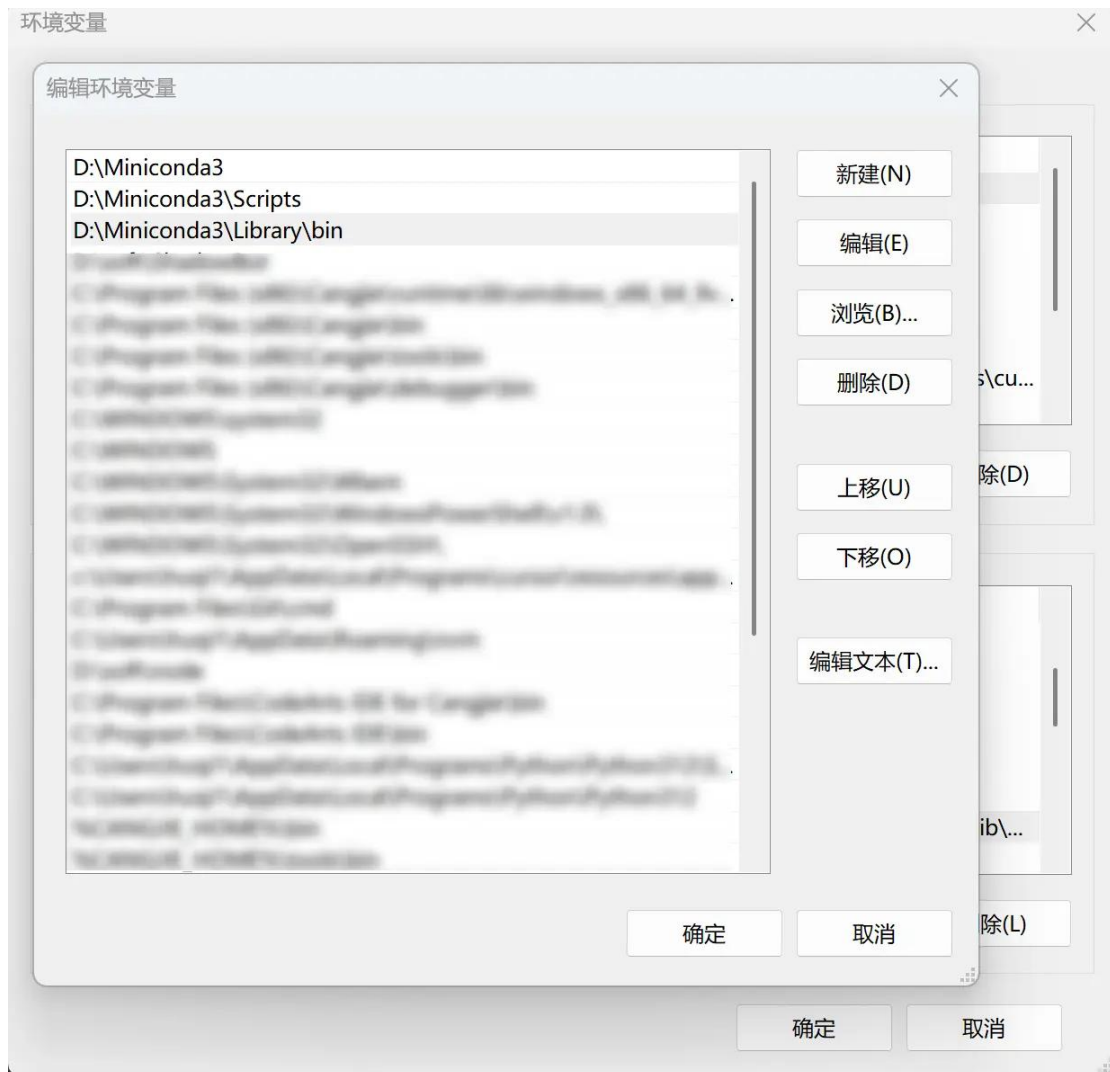


\* 在“编辑环境变量”窗口中，新建三个路径，将它们指向你 Miniconda 的安装目录下的相应文件夹。如果你的安装路径是 `D:\Miniconda3`，则需要添加：

D:\Miniconda3

D:\Miniconda3\Scripts

D:\Miniconda3\Library\bin



\* 完成后，一路点击“确定”保存更改。

#### 4. **\*\*配置 Conda 镜像源\*\***

为了加快后续使用 `conda` 安装包的速度，强烈建议配置国内镜像源。打开一个新的终端或 Anaconda Prompt，运行以下命令：

```
conda config --add channels
```

```
https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/main/
```

```
conda config --add channels
```

```
https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/free/
```

```
conda config --set show_channel_urls yes
```

配置完成后, 可以通过 `conda config --show channels` 命令查看已添加的源。

## 二、项目代码拉取

### 1. 安装 Git

如果你尚未安装 Git, 请按照以下步骤安装。

**\*\*Windows 系统\*\***: 访问[Git 官方网站](https://git-scm.com/download/win), 下载并运行安装程序, 按照默认设置完成安装。

**\*\*macOS 系统\*\***: 打开终端, 输入以下命令安装 Git:

```
brew install git
```

**\*\*Linux 系统 (以 Ubuntu 为例) \*\***: 打开终端, 输入以下命令安装 Git:

```
sudo apt-get update
```

```
sudo apt-get install git
```

安装完成后, 验证 Git 是否安装成功, 输入以下命令:

```
git --version
```

如果成功, 会显示 Git 的版本号。

### 2. 克隆项目代码

#### 1. **\*\*选择存放项目的目录\*\***

打开终端 (或 Windows 中的 Git Bash) , 导航到你想要存放项目的目录:

```
cd [你希望存放项目的路径]
```

## 2. **\*\*克隆仓库\*\***

使用以下命令拉取 `all-in-rag` 仓库：

```
git clone https://github.com/datawhalechina/all-in-rag.git
```

等待下载完成，项目代码将存放在当前目录下的 `all-in-rag` 文件夹中。

## 3. **\*\*进入项目目录\*\***

拉取代码后，进入项目目录：

```
cd all-in-rag
```

## 3. **创建并激活虚拟环境**

在项目目录下，推荐使用前面配置好的 Miniconda 来创建 Python 虚拟环境。

### 1. **\*\*创建虚拟环境\*\***

```
conda create --name all-in-rag python=3.12.7
```

### 2. **\*\*激活虚拟环境\*\***

所有系统统一使用以下命令激活虚拟环境：

```
conda activate all-in-rag
```

### 3. **\*\*依赖安装\*\***

如果严格安装上述流程当前应该在项目根目录，进入 code 目录安装依赖库

```
cd code
```

```
pip install -r requirements.txt
```

## 三、大模型 API 配置

### ### 1.1 AIHubmix API 申请（推荐）

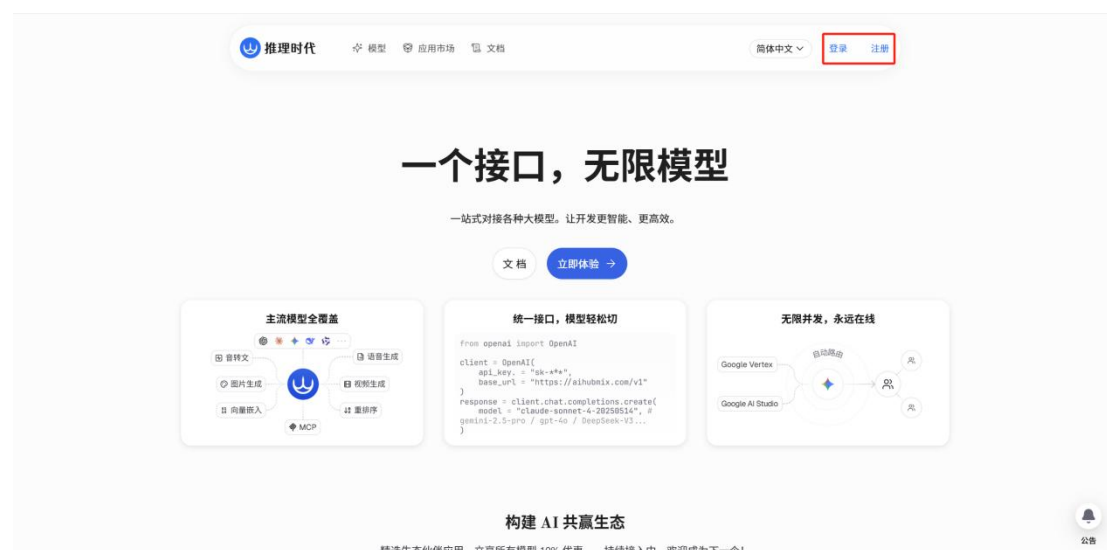
AIHubmix 是一个美国平台，公司注册在美国的特拉华州，一站式聚合了全球主流的 AI 模型，最新的模型通常能在发布当天最晚不超过 1 周就会支持。完全对接相关模型的云厂商（OpenAI 对接的是 Azure 云，Gemini 对接的 Google 官方，Claude 对接的是 AWS，其他开源等模型是对接到各大知名云厂商或者推理公司）。

AIHubmix 的服务器是在美国谷歌云上采用集群部署，同时因为完全对接云厂商，所以稳定性非常好，有多端点路由机制，可以达到比直连官方更稳定的效果。

> AIHubmix 提供的免费模型足够我们完成项目的学习。

#### 1. \*\*访问 AIHubmix 平台\*\*

访问 AIHubmix (<https://aihubmix.com/?aff=anNj>)。

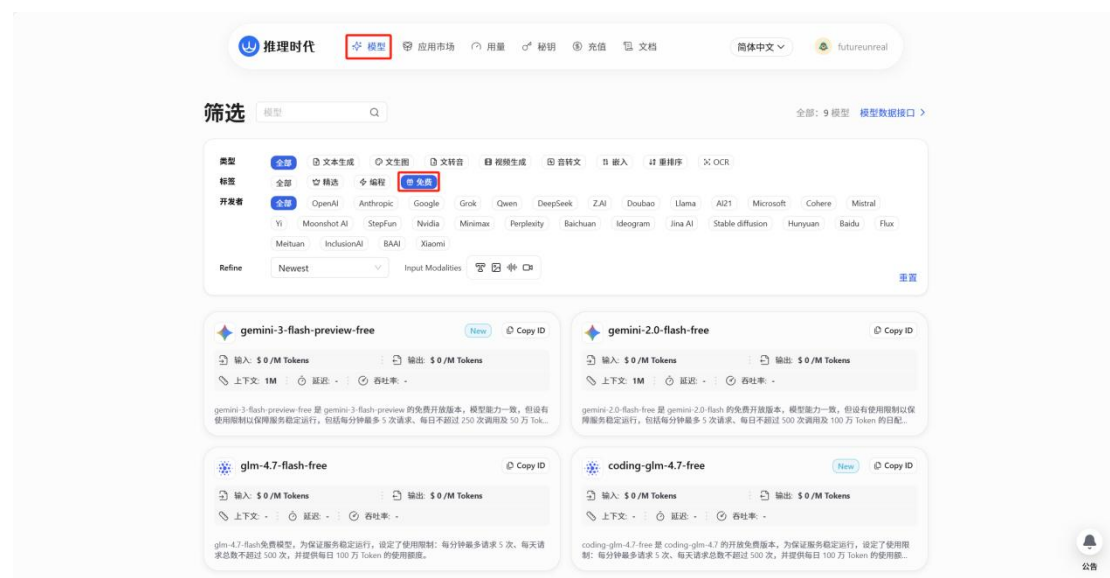


#### 2. \*\*登录或注册账号\*\*

如果已有账号，可以直接登录。如果没有，请点击页面右上角的注册按钮，使用邮箱或手机号完成注册。

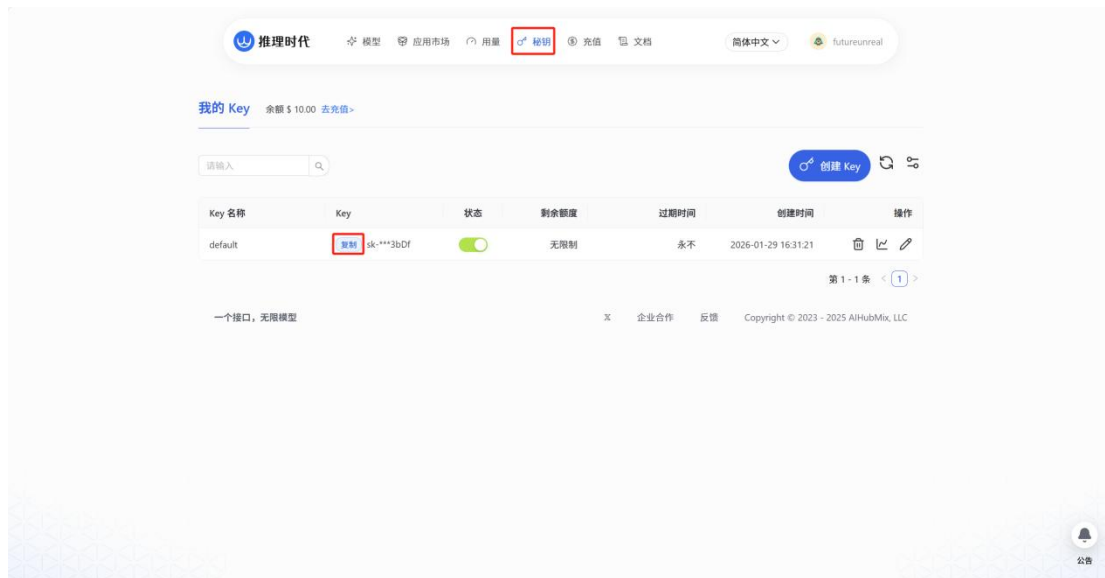
### 3. \*\*模型筛选\*\*

注册完成后，来到[模型页面](<https://aihubmix.com/models>)。标签选择`免费`，可以看到官方提供了一定数量的免费模型。而且 AIHubmix 还提供了很多嵌入和重排序的国内外模型选择，这些在 RAG 领域都很常用。



### 4. \*\*管理 API 密钥\*\*

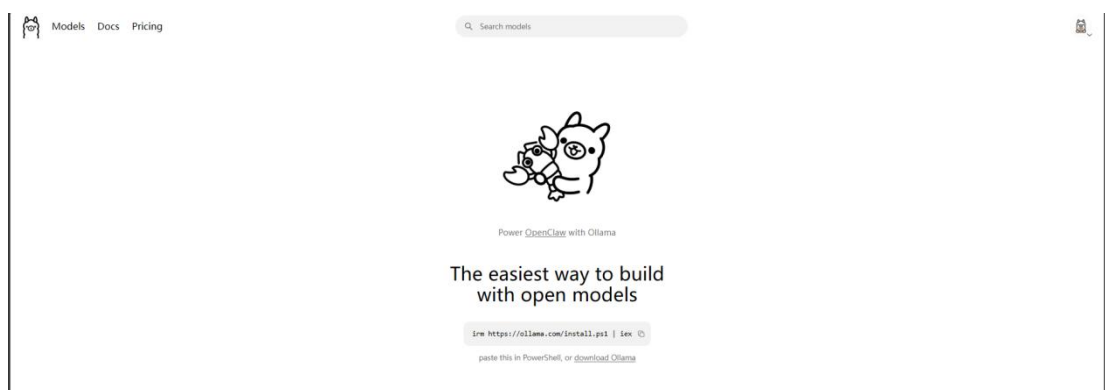
接着进入密钥管理页(<https://console.aihubmix.com/token>)，如下图所示，默认已经有了一个密钥可以直接复制使用。当然也可以点击创建 Key 填写名称后重新创建一个。



## ### 1.2 Ollama 配置 (推荐)

Ollama 是一个轻量级、可本地运行的大语言模型工具, 支持 Llama 3、Qwen、Gemma 等主流模型。它提供与 OpenAI 兼容的 API 接口, 无需联网即可调用, 非常适合隐私敏感或离线场景。同时, Ollama 也可以部署在云端服务器上, 实现团队共享或高并发访问。

### 1. \*\*安装 Ollama\*\*



- \*\*Windows\*\*: 访问 [Ollama 官网](<https://ollama.com/>), 下载 `OllamaSetup.exe` 并运行安装。安装完成后, Ollama 会自动在后台运行, 系统托盘区会出现羊驼图标。



- \*\*macOS\*\*: 使用 [Homebrew](https://brew.sh/) 安装:

```
```bash
```

```
brew install ollama
```

```
ollama serve
```

## 2. 模型部署

### 2.1 \*\*部署本地模型\*\*

打开控制台，下载并部署本地模型。

```
ollama pull deepseek-r1:7b
```

```
PS C:\Users\95737> ollama pull deepseek-r1:7b
pulling manifest
pulling 96c415656d37: 100%
pulling c5ad996bda6e: 100%
pulling 6e4c38e1172f: 100%
pulling f4d24e9138dd: 100%
pulling 40fb844194b2: 100%
verifying sha256 digest
writing manifest
success
```

打开控制台输入 `ollama list`，查看本地模型

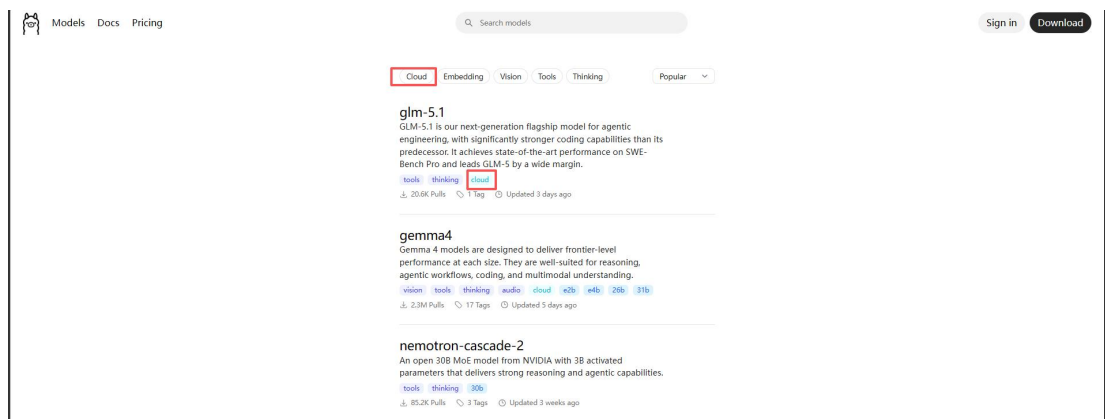
```
PS C:\Users\95737> ollama list
NAME                ID                SIZE      MODIFIED
deepseek-r1:7b      755ced02ce7b      4.7 GB    16 minutes ago
glm-5.1:cloud       59472abf9d0a      -         2 hours ago
```

### 2.2 \*\*部署 Ollama 云仓库模型\*\*

在官网注册账号，并且在本地登录即可使用云库模型



## 有 cloud 标签的模型可在云仓库使用



### 3. \*\*启动 Ollama\*\*

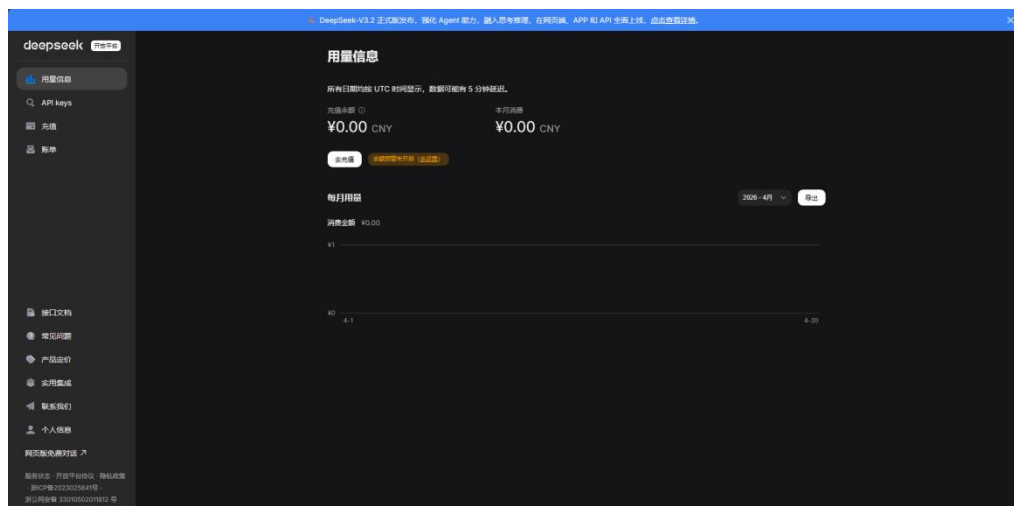
确定 ollama 打开之后(有羊驼图标)，将代码中 base\_url 设置为 **http://localhost:11434**，然后设置模型名字

## ### 1.3 DeepSeek API 申请

要使用 Deepseek 提供的大语言模型服务，你首先需要有一个 API Key。下面是申请步骤：

### 1. \*\*访问 Deepseek 开放平台\*\*

打开浏览器，访问 **https://platform.deepseek.com/**

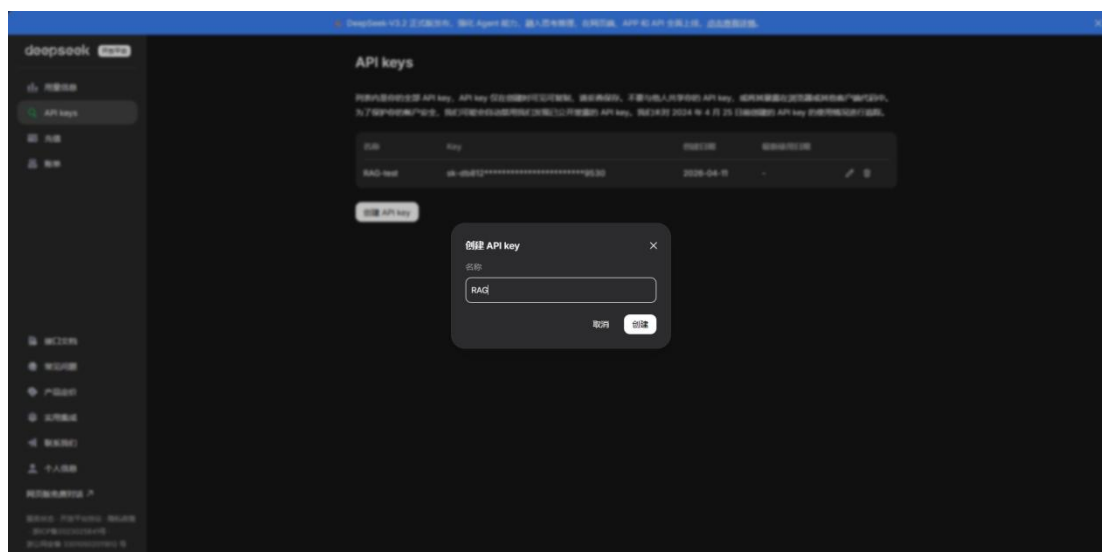


## 2. \*\*登录或注册账号\*\*

如果你已有账号, 请直接登录。如果没有, 请点击页面上的注册按钮, 使用邮箱或手机号完成注册。

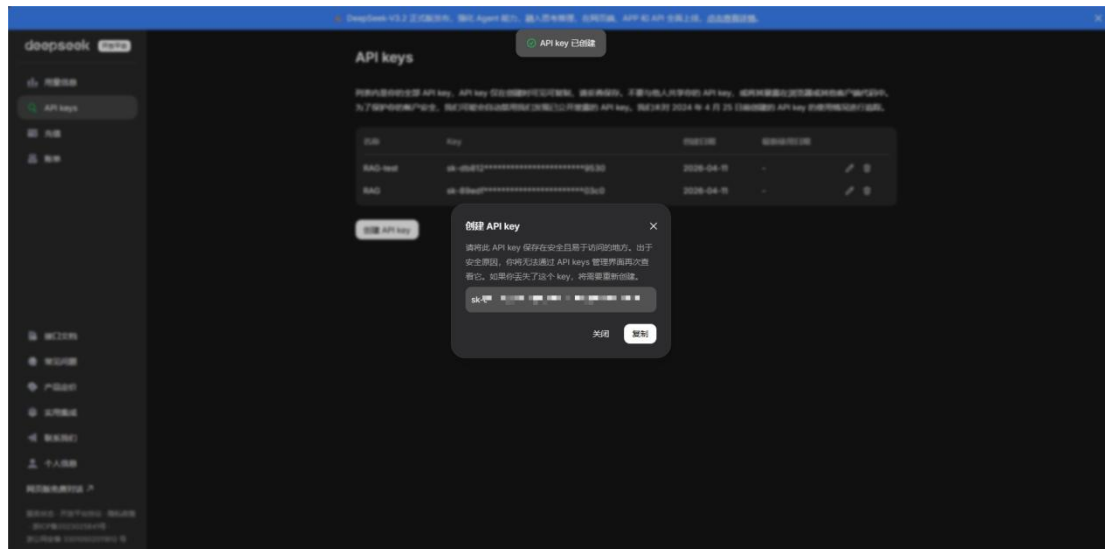
## 3. \*\*创建新的 API 密钥\*\*

登录成功后, 在页面左侧的导航栏中找到并点击 `API Keys`。在 API 管理页面, 点击 `创建 API key` 按钮。输入一个跟其他 api key 不重复的名称后点击创建。



## 4. \*\*保存 API Key\*\*

系统会为你生成一个新的 API 密钥。请\*\*立即复制\*\*并将其保存在一个安全的地方。 > 注意：出于安全原因，这个密钥只会完整显示一次，关闭弹窗后就没法再看到了。



```
# 使用 aihubmix
llm = ChatOpenAI(
    model="coding-glm-5.1-free",
    temperature=0.7,
    max_tokens=4096,
    api_key='sk-chACqWwyZYjfkW1gE1170b1dB44348E7A63882716c671aC0',
    base_url="https://aihubmix.com/v1")

# 使用 Ollama
llm = ChatOllama(
    model="glm-5.1:cloud",          #带 ":cloud" 后缀的模型名为云仓库模型
    temperature=0.7,
    num_predict=4096,
    base_url="http://localhost:11434", # Ollama 端口
    # 如果想覆盖默认的 HTTP 超时或请求头，可自行传入 `client_kwargs`
    # client_kwargs={"timeout": 60})

# 使用 deepseek api
llm = ChatOpenAI(
    model="deepseek-chat",
    temperature=0.7,
    max_tokens=4096,
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com")
```

## 四、构建 RAG

### 1. 启动虚拟环境

#### 1. **\*\*激活虚拟环境\*\***

假设已经按照前一章节的指导，创建了名为 `all-in-rag` 的 Conda 虚拟环境。在运行脚本前，先激活虚拟环境：

> 如果使用是 Cloud Studio，需要确认当前是否是用户环境，如果不是请运行 `su ubuntu` 切换到用户环境。 =

```
conda activate all-in-rag
```

#### ### 1.2 切换到项目目录

# 假设当前在 all-in-rag 项目的根目录下

```
cd code/C1
```

每章内容中的代码文件都存放在 `code/Cx` 目录下，其中 `x` 表示章节编号。

#### 2. **\*\*运行 RAG 示例代码\*\***

完成上述所有设置后，就可以运行 RAG 示例了。

打开终端，确保虚拟环境已激活，然后执行以下命令：

```
python 01_langchain_example.py
```

> 若出现 nltk 相关报错，尝试运行代码路径下

[fix\_nltk.py](https://github.com/datawhalechina/all-in-rag/blob/main/code/C1/fix\_nltk.py)

代码运行后，可以看到类似下面的输出（格式化后）：

```
Downloading Model from https://www.modelscope.cn to directory:
Path\to\all-in-rag\models\bge-small-zh-v1.5
2025-06-08 02:36:19,318 - modelscope - INFO - Target directory already exists,
skipping creation.
```

```
content='
```

文中举了以下例子：

1. **\*\*自然界中的羚羊\*\***：刚出生的羚羊通过试错学习站立和奔跑，适应环境。
2. **\*\*股票交易\*\***：通过买卖股票并根据市场反馈调整策略，最大化奖励。
3. **\*\*雅达利游戏（如 Breakout 和 Pong）\*\***：通过不断试错学习如何通关或赢得游戏。
4. **\*\*选择餐馆\*\***：利用（去已知喜欢的餐馆）与探索（尝试新餐馆）的权衡。
5. **\*\*做广告\*\***：利用（采取已知最优广告策略）与探索（尝试新广告策略）。
6. **\*\*挖油\*\***：利用（在已知地点挖油）与探索（在新地点挖油，可能发现大油田）。
7. **\*\*玩游戏（如《街头霸王》）\*\***：利用（固定策略如蹲角落出脚）与探索（尝试新招式如“大招”）。

这些例子用于说明强化学习中的核心概念（如探索与利用、延迟奖励等）及其在实际场景中的应用。

```
'
additional_kwargs={'refusal': None}
response_metadata={
    'token_usage': {
        'completion_tokens': 209,
        'prompt_tokens': 5576,
        'total_tokens': 5785,
        'completion_tokens_details': None,
        'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens':
5568},
        'prompt_cache_hit_tokens': 5568,
        'prompt_cache_miss_tokens': 8
    },
    'model_name': 'deepseek-chat',
    'system_fingerprint': 'fp_8802369eaa_prod0425fp8',
    'id': '67a0580d-78b1-44d6-bccf-f654ae0e9bba',
    'service_tier': None,
    'finish_reason': 'stop',
    'logprobs': None
}
id='run--919cedcd-771e-4aed-8dfd-cf436795792e-0'
usage_metadata={
    'input_tokens': 5576,
    'output_tokens': 209,
    'total_tokens': 5785,
    'input_token_details': {'cache_read': 5568},
    'output_token_details': {}
}
```

> 首次运行时，脚本会下载`BAAI/bge-small-zh-v1.5`嵌入模型。

## 输出参数解析：

```
- content: 这是最核心的部分，即大型语言模型（LLM）根据你的问题和提供的上下文生成的具体回答。  
- additional_kwargs: 包含一些额外的参数，在这个例子中是 {'refusal': None}，表示模型没有拒绝回答。  
- response_metadata: 包含了关于 LLM 响应的元数据。  
  - token_usage: 显示了本次调用消耗的 token 数量，包括完成（completion_tokens）、提示（prompt_tokens）和总量（total_tokens）。  
  - model_name: 使用的 LLM 模型名称，当前是 deepseek-chat。  
  - system_fingerprint, id, service_tier, finish_reason, logprobs: 这些是更详细的 API 响应信息，例如 finish_reason: 'stop' 表示模型正常完成了生成。  
- id: 本次运行的唯一标识符。  
- usage_metadata: 与 response_metadata 中的 token_usage 类似，提供了输入和输出 token 的统计。
```

## 3. 基于 LangChain 框架的 RAG 实现

在第一节中，我们提到四步构建最小可行系统分别是数据准备、索引构建、检索优化和生成集成。下面将围绕这四个方面来实现一个基于 LangChain 框架的 RAG 应用。

> [本节完整代

码]([https://github.com/datawhalechina/all-in-rag/blob/main/code/C1/01\\_langchain\\_example.py](https://github.com/datawhalechina/all-in-rag/blob/main/code/C1/01_langchain_example.py))

### 3.1 初始化设置

首先进行基础配置，包括导入必要的库、加载环境变量以及下载嵌入模型。

```
import os  
# os.environ['HF_ENDPOINT'] = 'https://hf-mirror.com'  
from dotenv import load_dotenv  
from langchain_community.document_loaders import TextLoader  
from langchain_text_splitters import RecursiveCharacterTextSplitter  
from langchain_huggingface import HuggingFaceEmbeddings  
from langchain_core.vectorstores import InMemoryVectorStore  
from langchain_core.prompts import ChatPromptTemplate
```

```
from langchain_deepseek import ChatOpenAI
from langchain_community.chat_models import ChatOllama#不用 ollama 可以不引入
# 加载环境变量
load_dotenv()
```

### ### 3.2 数据准备

- **加载原始文档**: 先定义 Markdown 文件的路径, 然后使用 `TextLoader` 加载该文件作为知识源。

```
markdown_path = "../data/C1/markdown/easy-r1-chapter1.md"
loader = TextLoader(markdown_path)
docs = loader.load()
```

- **文本分块 (Chunking)**: 为了便于后续的嵌入和检索, 长文档被分割成较小的、可管理的文本块 (chunks)。这里采用了递归字符分割策略, 使用其默认参数进行分块。当不指定参数初始化 `RecursiveCharacterTextSplitter()` 时, 其默认行为旨在最大程度保留文本的语义结构:

- **默认分隔符与语义保留**: 按顺序尝试使用一系列预设的分隔符 `["\n\n" (段落), "\n" (行), " " (空格), "" (字符)]` 来递归分割文本。这种策略的目的是尽可能保持段落、句子和单词的完整性, 因为它们通常是语义上最相关的文本单元, 直到文本块达到目标大小。

- **保留分隔符**: 默认情况下 (`keep_separator=True`), 分隔符本身会被保留在分割后的文本块中。

- **默认块大小与重叠**: 使用其基类 `TextSplitter` 中定义的默认参数 `chunk_size=4000` (块大小) 和 `chunk_overlap=200` (块重叠)。这些参数确保文本块符合预定的大小限制, 并通过重叠

来减少上下文信息的丢失。

```
text_splitter = RecursiveCharacterTextSplitter()
texts = text_splitter.split_documents(docs)
```

### ### 3.3 索引构建

数据准备完成后，接下来构建向量索引：

- **\*\*初始化中文嵌入模型\*\***：使用`HuggingFaceEmbeddings`加载之前在初始化设置中下载的中文嵌入模型。配置模型在 CPU 上运行，并启用嵌入归一化（`normalize\_embeddings: True`）。

```
embeddings = HuggingFaceEmbeddings(
    model_name="BAAI/bge-small-zh-v1.5",
    model_kwargs={'device': 'cpu'},
    encode_kwargs={'normalize_embeddings': True}
)
```

- **\*\*构建向量存储\*\***：将分割后的文本块（`texts`）通过初始化好的嵌入模型转换为向量表示，然后使用`InMemoryVectorStore`将这些向量及其对应的原始文本内容添加进去，从而在内存中构建出一个向量索引。

```
vectorstore = InMemoryVectorStore(embeddings)
vectorstore.add_documents(texts)
```

这个过程完成后，便构建了一个可供查询的知识索引。

### ### 3.4 查询与检索

索引构建完毕后，便可以针对用户问题进行查询与检索：

- **\*\*定义用户查询\*\***：设置一个具体的用户问题字符串。

```
question = "文中举了哪些例子？"
```

- **\*\*在向量存储中查询相关文档\*\***：使用向量存储的`similarity\_search`方法，根据用户问题在索引中查找最相关的`k`（此处示例中`k=3`）个文本块。

```
retrieved_docs = vectorstore.similarity_search(question, k=3)
```

- **\*\*准备上下文\*\***: 将检索到的多个文本块的页面内容 (`doc.page_content`) 合并成一个单一的字符串, 并使用双换行符 (`"\n\n"`) 分隔各个块, 形成最终的上下文信息 (`docs_content`) 供大语言模型参考。

```
docs_content = "\n\n".join(doc.page_content for doc in retrieved_docs)
```

> 使用 `"\n\n"` (双换行符) 而不是 `"\n"` (单换行符) 来连接不同的检索文档块, 主要是为了在传递给大型语言模型 (LLM) 时, 能够更清晰地在语义上区分这些独立的文本片段。双换行符通常代表段落的结束和新段落的开始, 这种格式有助于 LLM 将每个块视为一个独立的上下文来源, 从而更好地理解 and 利用这些信息来生成回答。

### ### 3.5 生成集成

最后一步是将检索到的上下文与用户问题结合, 利用大语言模型 (LLM) 生成答案:

- **\*\*构建提示词模板\*\***: 使用 `ChatPromptTemplate.from_template` 创建一个结构化的提示模板。此模板指导 LLM 根据提供的上下文 (`context`) 回答用户的问题 (`question`), 并明确指出在信息不足时应如何回应。

```
prompt = ChatPromptTemplate.from_template("""请根据下面提供的上下文信息来回答问题。

请确保你的回答完全基于这些上下文。

如果上下文中没有足够的信息来回答问题, 请直接告知: “抱歉, 我无法根据提供的上下文找到相关信息来回答此问题。”

上下文:
{context}
问题: {question}
回答: """)
```

- **\*\*配置大语言模型\*\***: 初始化 `ChatOpenAI` 客户端, 配置所用模型 (`glm-4.7-flash-free`)、生成答案的温度参数 (`temperature=0.7`)、最大 Token 数 (`max\_tokens=2048`) 以及 API 密钥 (从环境变量加载) 和 url。

```
#以 aihubmix 为例
llm = ChatOpenAI(
    model="glm-4.7-flash-free",
    temperature=0.7,
    max_tokens=2048,
    api_key=os.getenv("DEEPSEEK_API_KEY")
    base_url="https://aihubmix.com/v1"
)
```

- **\*\*调用 LLM 生成答案并输出\*\***: 将用户问题 (`question`) 和先前准备好的上下文 (`docs\_content`) 格式化到提示模板中, 然后调用 ChatDeepSeek 的 `invoke` 方法获取生成的答案。

```
answer = llm.invoke(prompt.format(question=question,
context=docs_content))
print(answer)
```